# SIMATS ENGINEERING

## ASSESSMENT TOOL 1: Algorithm Design Challenge

**COURSE NAME:** ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE:** MLA01

---

**COURSE OUTCOME COVERED**

**CO1:** *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems.* (BTL 3)

Assessment Tool Weightage: 40 %

---

**Total Marks: 100** **Time: 90 Mins**

**No. of Questions: 5**

Annexure A

## Algorithm Design Challenge

---

***Code of Conduct:***

*I _ S k . F a w a z A h a m a d / 1 9 2 4 2 5 3 9 0 ( (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

*Sk. Fawaz Ahamad*

***Signature of the student:***

| Criterion | Weightage | Marks Obtained |
|---|---|---|
| Problem Analysis and Understanding | 15% | |
| AI Algorithm Selection & Justification | 15% | |
| Algorithm Design / Pseudocode | 25% | |
| Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.) | 15% | |
| Diagram / State Space / Search Tree Representation | 10% | |
| Complexity Analysis and Solution Efficiency | 10% | |
| Originality, Critical Thinking, and Presentation | 10% | |
| Total | 100% | |

Signature of the Faculty Total Marks Awarded

## Answer all the Questions

| Q.No | Question | Topics |
|---|---|---|
| 1 | **Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given Romania Map Problem graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.** | Informed Search, Greedy Best-First Search, Heuristic Functions |
| 2 | **A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate g(n), h(n), and f(n) for each expanded node, and explain why A guarantees the optimal solution.** | A* Search, Heuristic Functions |
| 3 | **A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.** | CSP, MRV, Forward Checking |
| 4 | **Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha–Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.** | Game Playing, Minimax, Alpha–Beta Pruning |
| 5 | **A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.** | Local Search, Optimization, Online Search Agents |

**Structure of the Answer – Algorithm Design Challenge**

| S.No | Sections | Expected Content |
|---|---|---|
| 1 | **Problem Statement** | Briefly describe the given problem in your own words. |
| 2 | **Problem Analysis** | Identify the initial state, goal state, assumptions, and constraints. |
| 3 | **AI Technique Selection** | Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search). |
| 4 | **Justification of Algorithm** | Explain why the selected algorithm is suitable compared to other search techniques. |
| 5 | **State Space Representation** | Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable. |
| 6 | **Variables, Domains, and Constraints (Applicable for CSP problems)** | Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable". |
| 7 | **Heuristic Function (Applicable for Informed Search)** | Define the heuristic function h(n) and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable". |
| 8 | **Algorithm Design / Pseudocode** | Write the step-by-step algorithm or pseudocode for solving the problem. |
| 9 | **Flowchart / Algorithm Diagram** | Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution. |
| 10 | **Working / Execution Trace** | Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation. |
| 11 | **Complexity Analysis** | Analyze the Time Complexity and Space Complexity of the proposed algorithm. |
| 12 | **Advantages and Limitations** | Discuss the strengths and limitations of the selected algorithm for the given problem. |
| 13 | **Conclusion** | Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches. |

## Q1. Greedy Best-First Search – Romania Map Problem

### 1. Problem Statement

The Romania Map Problem is a classical state-space search problem in Artificial Intelligence. The objective is to find a path from a source city to a destination city using **Greedy Best-First Search (GBFS)**.

For this solution, consider **Arad as the source** and **Bucharest as the goal**. Greedy Best-First Search selects the node that appears closest to the goal based only on the heuristic value.

### 2. Problem Analysis

- **Initial State:** Arad
- **Goal State:** Bucharest
- **State:** Each city in the Romania map.
- **Action:** Travel from one connected city to another.
- **Path Cost:** Distance between cities.
- **Constraint:** Movement is possible only along connected roads.
- **Objective:** Reach Bucharest using the node with the smallest heuristic value.

### 3. AI Technique Selection

The selected technique is **Greedy Best-First Search**.

GBFS uses:

where:

- = estimated cost from node to the goal.
- = priority value of the node.

For the Romania map, the heuristic is generally the **straight-line distance to Bucharest**.

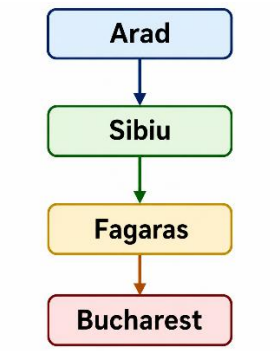
### 4. Justification of Algorithm

Greedy Best-First Search is suitable because it uses heuristic information to guide the search toward Bucharest instead of exploring nodes randomly.

It is generally faster than uninformed methods when a good heuristic is available. However, it does **not guarantee the shortest path**, because it considers only and ignores the distance already travelled.

**5. State Space Representation**

A simplified search path is:




**Heuristic values:**

| **Node** | **h(n) to Bucharest** |
|---|---|
| Arad | 366 |
| Sibiu | 253 |
| Fagaras | 176 |
| Bucharest | 0 |

**6. Variables, Domains and Constraints**

**Not Applicable** because this is an informed search problem rather than a CSP.

**7. Heuristic Function**

The heuristic function is:

For example:

The heuristic is appropriate because it gives a useful estimate of the remaining distance.

**8. Algorithm / Pseudocode**

**GreedyBestFirstSearch(start, goal)**

1. Create OPEN priority queue.

2. Insert start node into OPEN.

3. Create CLOSED set.

4. While OPEN is not empty:

    a. Select node n having minimum h(n).

    b. If n = goal:

        return path.

    c. Move n from OPEN to CLOSED.

    d. Generate all successors of n.

    e. For every successor:

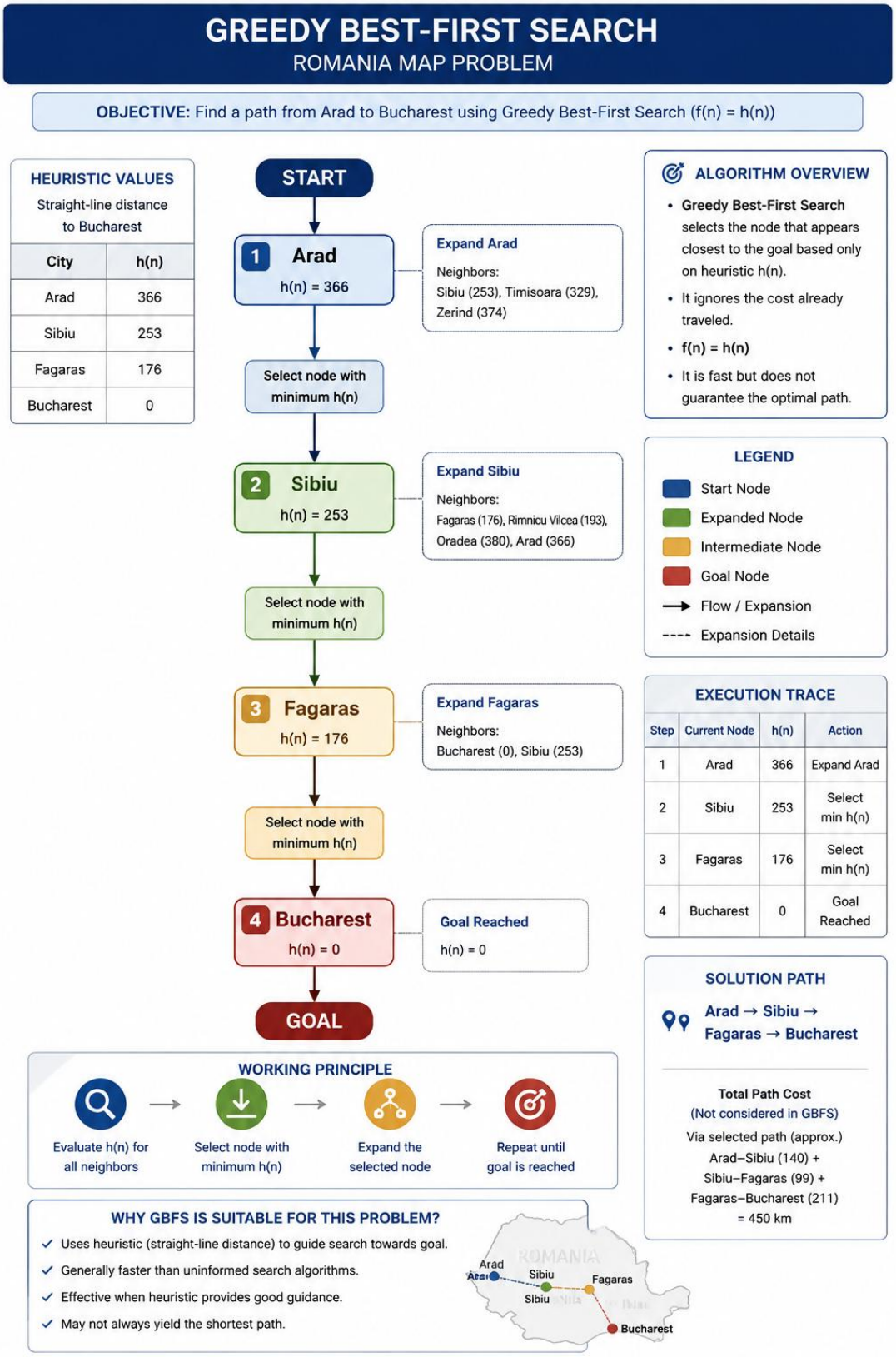
        If it is not in CLOSED:

            calculate h(successor)

            insert into OPEN.

5. Return failure if goal is not reached.

## 9. Search/Algorithm Diagram



**10. Working / Execution Trace**

| Step | Current Node | h(n) | Action |
|---|---|---|---|
| 1 | Arad | 366 | Expand Arad |
| 2 | Sibiu | 253 | Select smallest h |
| 3 | Fagaras | 176 | Select smallest h |
| 4 | Bucharest | 0 | Goal reached |

**Therefore, the obtained path is:**

$$Arad \rightarrow Sibiu \rightarrow Fagaras \rightarrow Bucharest$$

**11. Complexity Analysis**

If is the branching factor and is the depth of the solution:

- Time Complexity: in the worst case.
- Space Complexity: .

The actual performance depends strongly on the quality of the heuristic.

**12. Advantages and Limitations**

**Advantages**

- Simple and easy to implement.
- Uses heuristic knowledge.
- Usually reaches the goal faster than uninformed search.
- Useful for large search spaces.

**Limitations**

- Does not guarantee an optimal solution.
- Can be misled by an inaccurate heuristic.
- May explore an undesirable path because path cost is ignored.

### 13. Conclusion

Greedy Best-First Search provides an efficient heuristic-guided approach for reaching Bucharest. It prioritizes nodes that appear closer to the goal. Although it can be faster, it does not guarantee the shortest path. A* would be preferred when optimality is required.

## Q2. A* Search – Delivery Robot

### 1. Problem Statement

A delivery robot must travel from a warehouse to a customer through a weighted road network. The objective is to determine the **least-cost path** using the **A*** search algorithm.

The A* algorithm combines the actual cost already travelled with the estimated cost remaining.

### 2. Problem Analysis

- **Initial State:** Warehouse
- **Goal State:** Customer
- **States:** Road-network locations.
- **Actions:** Move between connected locations.
- **Cost:** Road distance/travel cost.
- **Goal:** Find the minimum-cost route from to .

### 3. AI Technique Selection

The selected algorithm is **A*** Search.

A* uses:

where:

- = actual cost from start to node .
- = estimated cost from to goal.
- = estimated total solution cost.

### 4. Justification

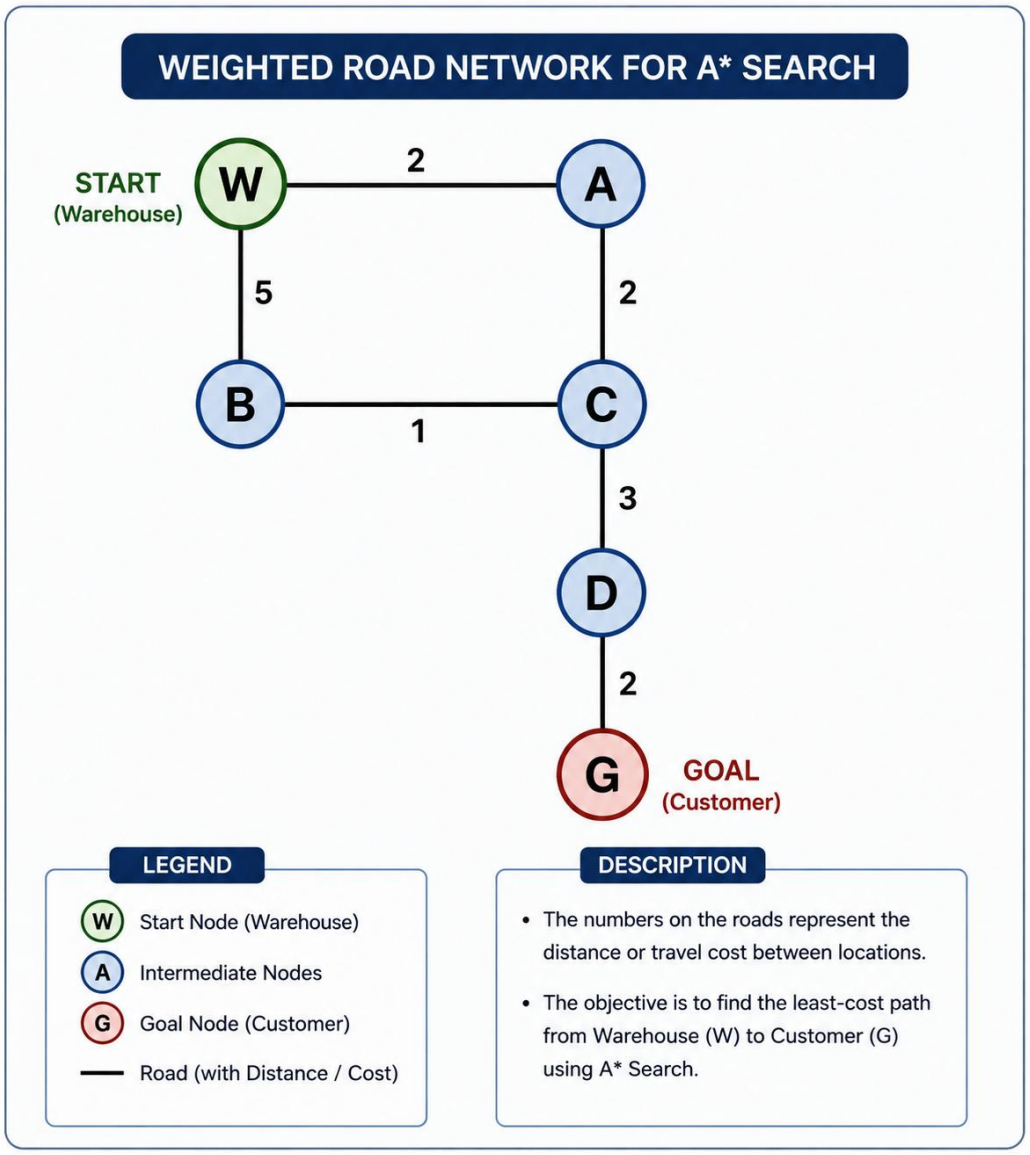
A* is suitable because the problem involves:

- Weighted paths.
- A clearly defined goal.
- A heuristic estimate.
- Requirement for an optimal solution.

If the heuristic is **admissible**, meaning it never overestimates the actual remaining cost, A* can guarantee an optimal solution.

**5. State Space Representation**

An illustrative road network is:



**6. Variables, Domains and Constraints**

**Not Applicable** because this is an informed search problem.

**7. Heuristic Function**

Use an admissible estimate of the remaining travel cost:

| **Node** | **h(n)** |
|---|---|
| W | 7 |
| A | 5 |
| B | 5 |
| C | 5 |
| D | 2 |
| G | 0 |

The heuristic does not overestimate the actual remaining cost.

**8. A* Pseudocode**

AStar(start, goal)

1. Put start into OPEN.

2. Set g(start) = 0.

3. Calculate f(start) = g(start) + h(start).

4. While OPEN is not empty:

    a. Select node n with minimum f(n).

    b. If n = goal:

        return optimal path.

    c. Move n to CLOSED.

    d. Generate successors of n.

e. For every successor:

calculate tentative g value.

if new path is better:

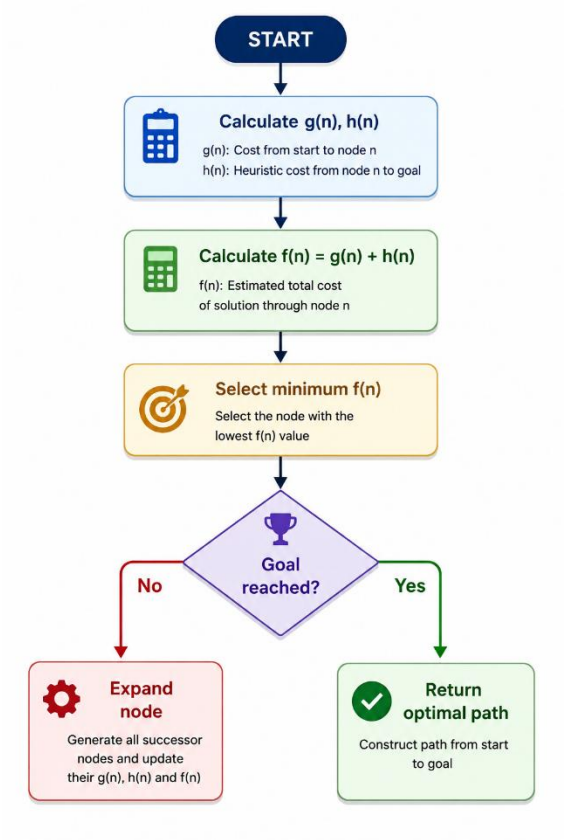
update g(n)

calculate f(n)

store parent.

5. Return failure if goal is not found.

**9. Algorithm Diagram**




**10. Working / Execution Trace**

| Node | g(n) | h(n) | f(n)=g+h |
|---|---|---|---|
| W | 0 | 7 | 7 |
| A | 2 | 5 | 7 |

| | | | |
|---|---|---|---|
| B | 5 | 5 | 10 |
| C | 4 | 5 | 9 |
| D | 7 | 2 | 9 |
| G | 9 | 0 | 9 |

**The optimal path is:**

$$W \rightarrow A \rightarrow C \rightarrow D \rightarrow G$$

**Total cost:**

2 + 2 + 3 + 2 = 9

**11. Complexity Analysis**

For a general search tree:

- **Time:** in the worst case.
- **Space:**.

With a good heuristic, A* can significantly reduce the number of nodes explored.

**12. Advantages and Limitations**

**Advantages**

- Finds an optimal solution with an admissible heuristic.
- Combines actual and estimated cost.
- More efficient than uninformed search in many problems.

**Limitations**

- Can require large memory.
- Performance depends on heuristic quality.
- Computationally expensive for very large search spaces.

**13. Conclusion**

A* is an effective algorithm for delivery-route optimization because it considers both the distance already travelled and the estimated remaining distance.

## Q3. CSP – University Examination Timetable

The PDF specifically asks for variables, domains, constraints and the use of **MRV and Forward Checking** for this question.

### 1. Problem Statement

A university needs to create an examination timetable in which subjects having common students must not be scheduled at the same time.

The problem can be represented as a **Constraint Satisfaction Problem (CSP)**.

### 2. Problem Analysis

The CSP consists of:

- **Variables:** Subjects.
- **Domains:** Available examination slots.
- **Constraints:** Conflicting subjects cannot share the same slot.
- **Goal:** Assign a valid time slot to every subject.

Consider five subjects:

$$\{AI, DBMS, CN, OS, ML\}$$

Available slots:

$$\{S1, S2, S3\}$$

### 3. AI Technique Selection

The selected technique is:

**CSP + MRV + Forward Checking**

CSP models the timetable problem, while MRV and Forward Checking improve the search process.

### 4. Justification

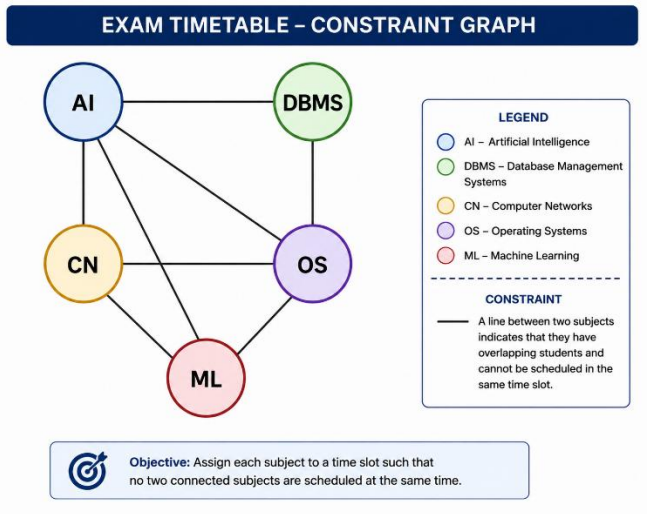
**MRV (Minimum Remaining Values)** selects the variable having the smallest number of available values.

**Forward Checking** removes invalid values from the domains of unassigned variables after every assignment.

Together, they reduce unnecessary search and detect conflicts earlier.

**5. Constraint Graph**

Assume the following subjects have conflicts:




A connection means that the two subjects cannot be placed in the same slot.

**6. Variables, Domains and Constraints**

**Variables:**

$$X = \{AI, DBMS, CN, OS, ML\}$$

**Domains:**

$$D(X) = \{S1, S2, S3\}$$

**Constraints:**

$$AI \neq DBMS$$

$$AI \neq CN$$

$$AI \neq ML$$

$$DBMS \neq OS$$

CN□=OS C

N□=ML

OS□=ML

**7. Heuristic Function**

**Not Applicable** in the traditional informed-search sense.

However, MRV acts as a variable-selection heuristic in CSP.

**8. Algorithm / Pseudocode**

**CSP-Timetable (Assignment)**

1. If all subjects are assigned:

    return Assignment.

2. Select an unassigned subject using MRV.

3. For each available slot:

    a. Check consistency with existing assignments.

    b. Assign the slot.

    c. Apply Forward Checking.

    d. If no domain becomes empty:

        recursively continue.

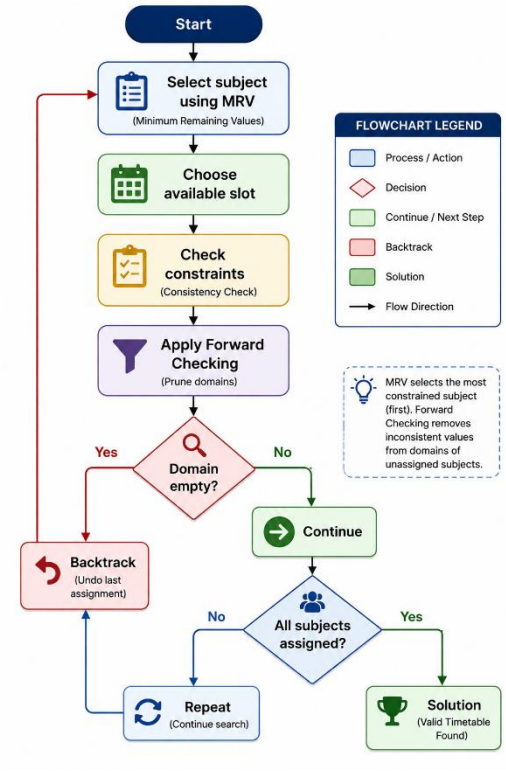
    e. Otherwise, remove the assignment.

4. If a valid assignment is found:

    return solution.

5. Return failure.

## 9. Workflow




## 10. Working / Execution Trace

An example valid timetable could be:

| Subject | Slot |
|---|---|
| **AI** | **S1** |
| **DBMS** | **S2** |
| **CN** | **S2** |
| **OS** | **S1** |
| **ML** | **S3** |

MRV helps choose the most constrained subject first.

Forward Checking removes the selected slot from the domains of conflicting subjects.

If a subject has no remaining valid slot, the algorithm immediately backtracks instead of continuing with an invalid partial timetable.

## 11. Complexity Analysis

For subjects and possible slots:

- Worst-case time:
- Space: approximately for the assignment and domains.

MRV and Forward Checking can considerably reduce practical search.

## 12. Advantages and Limitations

### Advantages

- Systematically handles constraints.
- MRV reduces unnecessary branching.
- Forward Checking detects conflicts early.
- Suitable for timetable and scheduling problems.

### Limitations

- Large CSPs can still have exponential search.
- Performance depends on constraint structure.
- Requires correct identification of conflicts.

## 13. Conclusion

The examination timetable can be efficiently formulated as a CSP. MRV selects the most constrained subject, while Forward Checking eliminates invalid future choices. This combination reduces unnecessary search and produces a valid timetable efficiently.

## Q4. Minimax and Alpha–Beta Pruning

### 1. Problem Statement

In a two-player competitive game, the MAX player attempts to maximize the utility value, while the MIN player attempts to minimize it.

The Minimax algorithm evaluates the game tree and determines the best move for MAX.

### 2. Problem Analysis

- Initial State: Root game state.
- Players: MAX and MIN.
- Terminal State: Leaf node.
- Utility: Numerical value at terminal nodes.
- Goal: MAX selects the move that gives the highest guaranteed utility.

### 3. AI Technique Selection

The selected algorithms are:

1. Minimax
2. Alpha–Beta Pruning

### 4. Justification

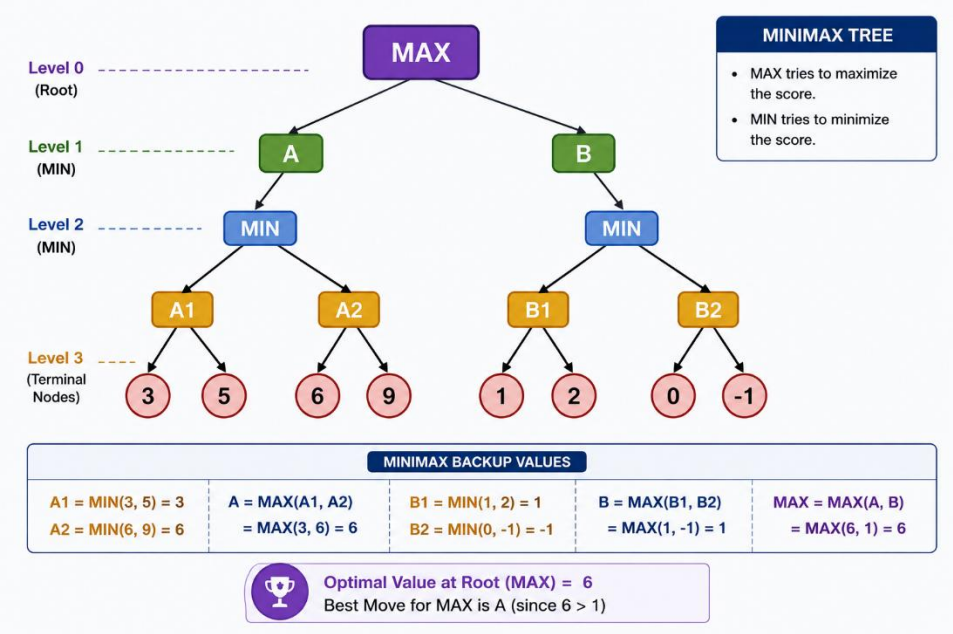
Minimax is appropriate for deterministic, two-player, zero-sum games.

Alpha–Beta pruning improves Minimax by eliminating branches that cannot influence the final decision.

It produces the same optimal move as Minimax but evaluates fewer nodes**.**

## 5. Game Tree Representation

**Consider:**




## 6. Variables, Domains and Constraints

Not Applicable because this is a game-playing problem.

## 7. Heuristic Function

Not Applicable for the given terminal-node evaluation.

## 8. Minimax Pseudocode

MINIMAX(node)

1. If node is terminal:

    return utility(node).

2. If node is MAX:

    return maximum value of children.

3. If node is MIN:

    return minimum value of children.

**9. Alpha–Beta Pseudocode**

ALPHA-BETA(node, α, β)

1. If node is terminal:

    return utility(node).

2. If node is MAX:

    value = -∞

    For each child:

        value = max(value,

            ALPHA-BETA(child, α, β))

        α = max(α, value)

        If α >= β:

            prune remaining children

    return value.

3. If node is MIN:

    value = +∞

    For each child:

        value = min(value,

            ALPHA-BETA(child, α, β))

        β = min(β, value)

        If α >= β:

            prune remaining children

    return value.

**10. Working / Execution Trace**

For branch A:

Therefore:

Now MAX has:

For branch B:

Since B is a MIN node and:

while:

we have:

Therefore, the remaining B2 branch can be pruned.

**Hence:**

B=1

**Finally:**

MAX=max (3,1) =3

**Therefore, MAX selects:**

Move A

**Node Comparison**

| **Method** | **Leaf Nodes Evaluated** |
|---|---|
| **Minimax** | **8** |
| **Alpha–Beta** | **5** |
| **Pruned** | **3** |

Thus Alpha–Beta pruning avoids evaluating unnecessary branches.

**11. Complexity Analysis**

For Minimax:

O(bd)

where:

- = branching factor.
- = depth.

Alpha–Beta:

- **Worst case:**
- **Best case:** approximately

Therefore, good move ordering can significantly improve efficiency.

**12. Advantages and Limitations**

**Minimax Advantages**

- Guarantees optimal decision against an optimal opponent.
- Simple conceptual model.

**Alpha–Beta Advantages**

- Evaluates fewer nodes.
- Produces the same result as Minimax.
- Allows deeper search within the same computational resources.

**Limitations**

- Large game trees can still be computationally expensive.
- Performance depends on branching factor and moves ordering.

**13. Conclusion**

Minimax determines that **Move A** is the best decision for MAX in the illustrative tree. Alpha–Beta pruning reduces unnecessary node evaluation without changing the final decision. Hence, Alpha–Beta is a more efficient implementation of Minimax.

## Q5. Dynamic Warehouse Robot – Online Search Agent

### 1. Problem Statement

A warehouse robot must move from a starting location to a target location. However, obstacles such as boxes, vehicles, or blocked paths may appear unexpectedly.

Therefore, the robot must continuously observe its environment and modify its route.

### 2. Problem Analysis

- Initial State: Robot's current position.
- Goal State: Required warehouse location.
- Environment: Dynamic.
- Actions: Move up, down, left, right.
- Problem: Obstacles may appear after planning.
- Objective: Reach the goal safely and efficiently.

### 3. AI Technique Selection

**The selected technique is an:**

Online Search Agent

An online search agent is suitable because the robot does not have complete information about future environmental changes.

### 4. Justification

A traditional informed search algorithm generally assumes that the environment is known before planning.

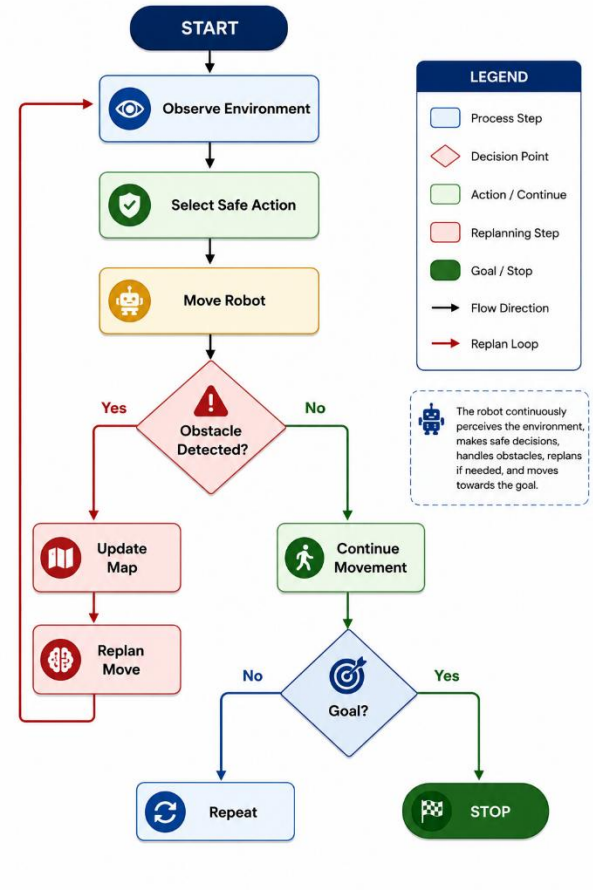
In contrast, an online agent:

1. Observes the current environment.
2. Takes an action.
3. Receives new information.
4. Updates its knowledge.

5. Replans when necessary.

This makes it suitable for dynamic warehouse environments.

**5. State-Space Representation**




**6. Variables, Domains and Constraints**

**Variables:**

- Robot position.
- Available movement directions.
- Obstacle locations.

**Domains:**

**Constraints:**

- Robot cannot move through obstacles.
- Robot must remain inside warehouse boundaries.

- Robot should avoid unsafe paths.
- Goal must eventually be reached.

**7. Heuristic Function**

For grid-based movement, Manhattan distance can be used:

$$h(n)=|xn−xg|+|yn−yg|$$

where:

- = current position.
- = goal position.

The heuristic estimates the distance to the target.

**8. Online Search Pseudocode**

**ONLINE-ROBOT-SEARCH(start, goal)**

1. Initialize current_position = start.

2. Observe the environment.

3. While current_position != goal:

   a. Detect obstacles.

   b. Update the environment map.

   c. Generate safe neighbouring states.

   d. Estimate distance to goal.

   e. Select the best safe action.

   f. Move robot.

   g. Observe the environment again.

4. Stop when goal is reached.

5. If the selected path becomes blocked:

   update map and replan.

## 9. Working Example

### 9. WORKING EXAMPLE FOR Q5

**Problem:** Find the shortest path from W to G.

| Iteration | Node Selected | g(n) | h(n) | f(n)=g(n)+h(n) | Path |
|---|---|---|---|---|---|
| Start | W | 0 | 4 | 4 | W |
| 1 | A | 2 | 3 | 5 | W → A |
| 2 | C | 4 | 2 | 6 | W → A → C |
| 3 | D | 7 | 1 | 8 | W → A → C → D |
| 4 | G (Goal) | 9 | 0 | 9 | W → A → C → D → G |

**Step-by-Step Explanation:**

1. **Start at W:** g(W) = 0 (start), h(W) = 4 (estimated to goal G), f(W) = 4.
2. **From W, choose A:** Edge W → A has cost 2 ⇒ g(A) = 2.
   h(A) = 3 ⇒ f(A) = 2 + 3 = 5.
3. **From A, choose C:** Edge A → C has cost 2 ⇒ g(C) = 2 + 2 = 4.
   h(C) = 2 ⇒ f(C) = 4 + 2 = 6.
4. **From C, choose D:** Edge C → D has cost 3 ⇒ g(D) = 4 + 3 = 7.
   h(D) = 1 ⇒ f(D) = 7 + 1 = 8.
5. **From D, choose G:** Edge D → G has cost 2 ⇒ g(G) = 7 + 2 = 9.
   h(G) = 0 ⇒ f(G) = 9 + 0 = 9.

**Optimal Path Found:**
**W → A → C → D → G with total cost = 9**

## 10. Execution Trace

| Step | Robot Position | Observation | Action |
|---|---|---|---|
| 1 | Start | Path clear | Move forward |
| 2 | A | Path clear | Move toward B |
| 3 | A | B blocked | Replan |
| 4 | D | Alternative path clear | Continue |
| 5 | E | Path clear | Continue |
| 6 | Goal | Destination reached | Stop |

## 11. Complexity Analysis

For a grid containing reachable states:

- Worst-case search time: per complete replanning/search operation.
- Space: for storing the explored map and search information.

In a dynamic environment, repeated replanning can increase computational cost

## 12. Comparison with Informed Search

| **Feature** | **Informed Search** | **Online Search** |
|---|---|---|
| Environment | Usually known | Partially/initially unknown |
| Planning | Before movement | During movement |
| Adaptability | Limited | High |
| Dynamic obstacles | Less suitable | Highly suitable |
| Replanning | Usually not continuous | Continuous |
| Example | A* | Online Search Agent |

## 13. Advantages and Limitations

### Advantages

- Adapts to unexpected obstacles.
- Uses newly acquired information.
- Suitable for real-world robots.
- Can replan during operation.

### Limitations

- Replanning requires computation.
- May not always produce the globally optimal route.

- Performance depends on sensor accuracy.
- Dynamic environments can cause repeated route changes.

**14. Conclusion**

An Online Search Agent is highly suitable for warehouse robots operating in dynamic environments. It continuously senses the environment, updates its knowledge and replans when obstacles appear. Compared with conventional informed search, online search provides better adaptability and practical robustness.

## EVALAUTION RUBRICS

| Criteria | Weightage (%) | Excellent (90–100%) | Good (75–89%) | Average (60–74%) | Poor (<60%) |
|---|---|---|---|---|---|
| Problem Analysis and Understanding | 15% | 13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome. | 10–12% – Identifies most problem components with minor omissions. | 7–9% – Demonstrates partial understanding with some missing elements. | 0–6% – Incorrect or incomplete understanding of the problem. |
| AI Algorithm Selection & Justification | 15% | 13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification. | 10–12% – Selects an appropriate algorithm with adequate justification. | 7–9% – Selects a partially suitable algorithm with limited justification. | 0–6% – Incorrect algorithm selection or no justification. |
| Algorithm Design / Pseudocode | 25% | 22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode. | 17–21% – Algorithm is mostly correct with minor logical errors. | 12–16% – Algorithm is partially complete with several logical errors. | 0–11% – Algorithm is incomplete, incorrect, or difficult to understand. |
| Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.) | 15% | 13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution. | 10–12% – Applies AI concepts correctly with minor mistakes. | 7–9% – Demonstrates limited application of AI concepts. | 0–6% – Unable to apply the appropriate AI concepts or techniques. |

| | | | | | |
|---|---|---|---|---|---|
| Diagram / State Space / Search Tree Representation | 10% | 9–10% – Accurate, complete, and well-labelled diagram supporting the solution. | 7–8% – Diagram is mostly correct with minor omissions. | 5–6% – Diagram is partially complete or lacks clarity. | 0–4% – Diagram is missing or incorrect. |
| Complexity Analysis and Solution Efficiency | 10% | 9–10% – Correctly analyzes time and space complexity and proposes an efficient solution. | 7–8% – Complexity analysis is mostly correct with minor errors. | 5–6% – Partial complexity analysis or limited discussion of efficiency. | 0–4% – No meaningful complexity analysis or highly inefficient solution. |
| Originality, Critical Thinking, and Presentation | 10% | 9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions. | 7–8% – Demonstrates good critical thinking with a clear presentation. | 5–6% – Shows limited originality and acceptable presentation. | 0–4% – Poor presentation with little or no critical thinking. |